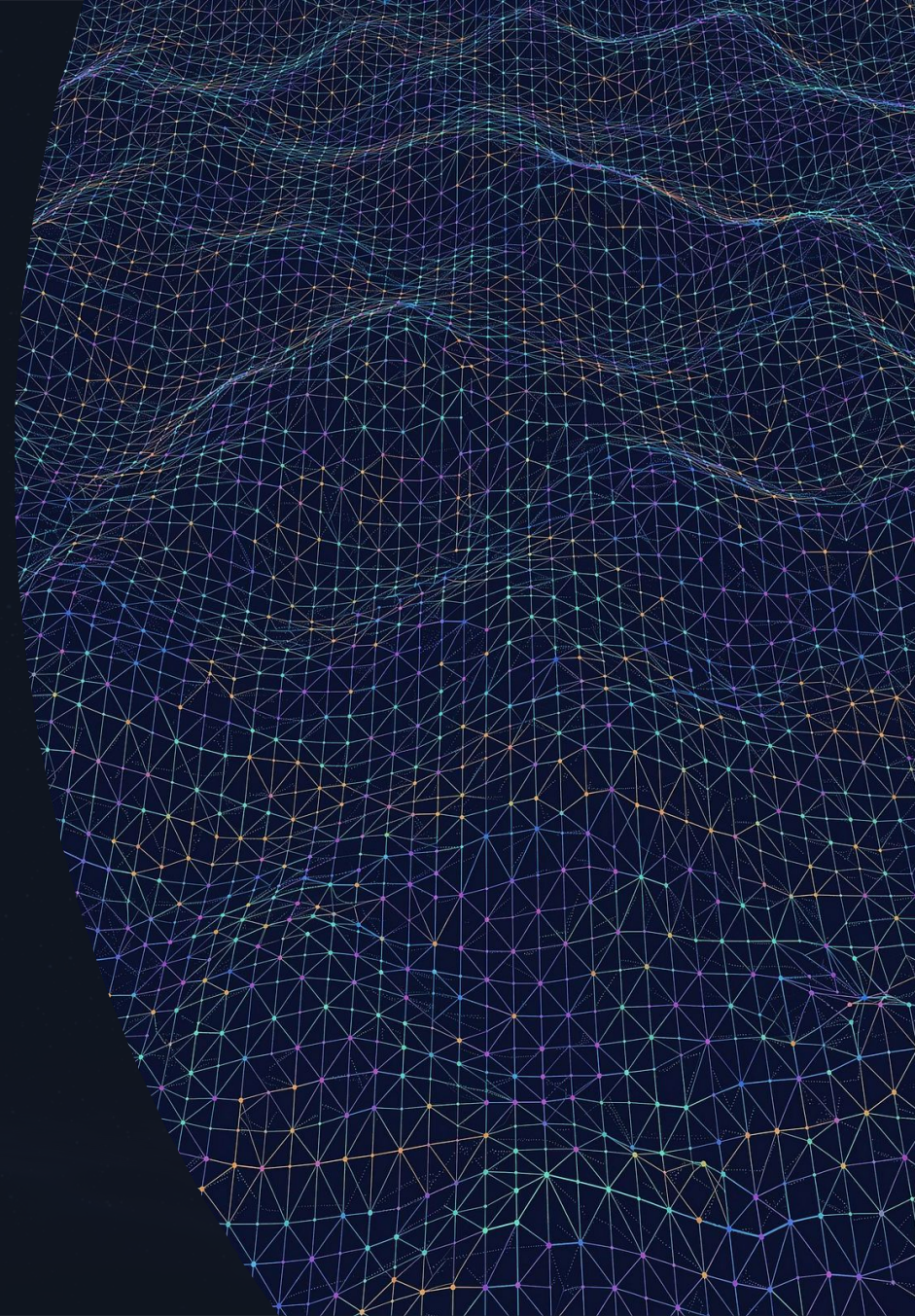


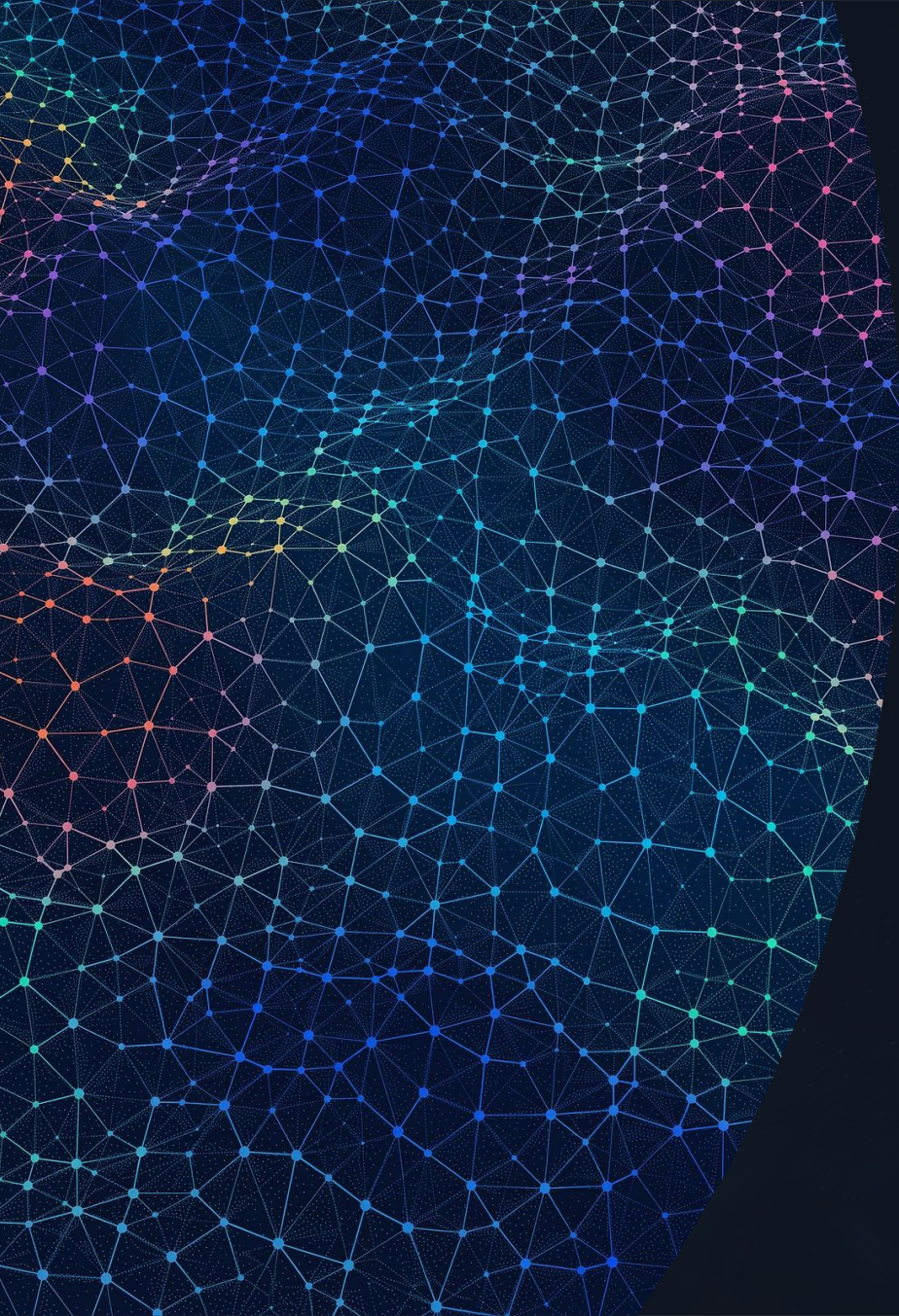
Puntos de Función

INGENIERÍA DE SOFTWARE III · UNIDAD 3

Medir funcionalidad para estimar esfuerzo — antes de escribir una sola línea de código.

UNPSJB · Facultad de Ingeniería · Sede Trelew





EL PROBLEMA CENTRAL

La pregunta imposible

¿Cuántas líneas de código va a tener tu sistema?

Nadie lo sabe. Y sin embargo hay que dar un plazo y un precio **antes** de escribir la primera línea. Ese es el problema real de la estimación de software.

EL GIRO

La pregunta que sí se puede contestar

Hoy, con el enunciado en la mano:

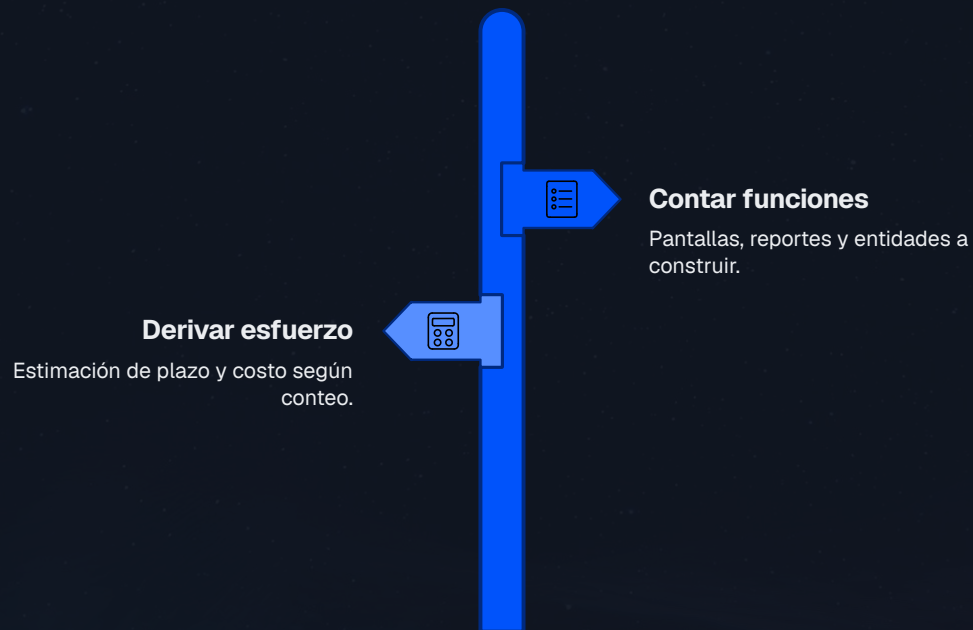
- ¿Cuántas pantallas de carga tiene?
- ¿Cuántos reportes genera?
- ¿Cuántas entidades mantiene?

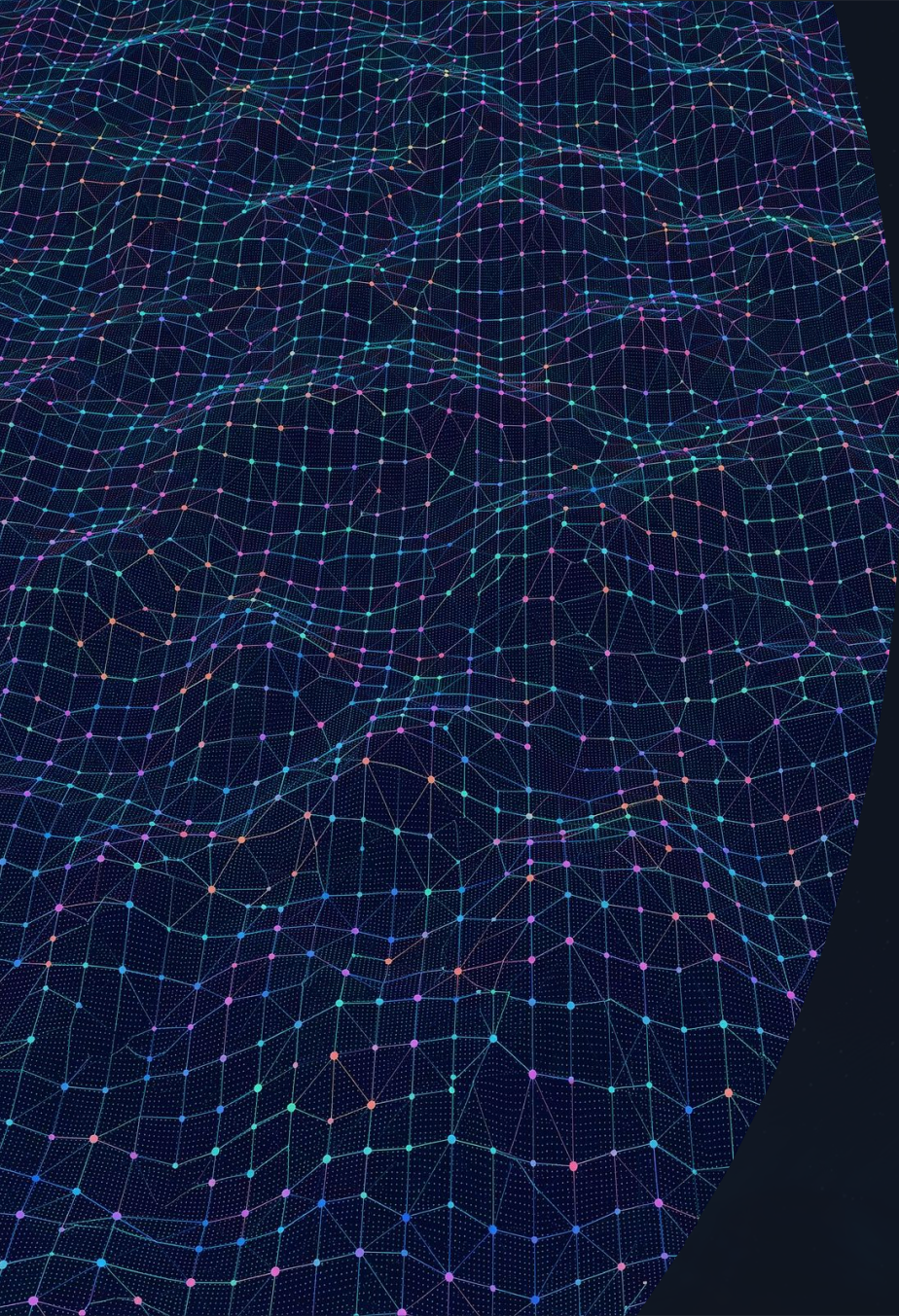
Esas preguntas **sí se pueden contestar ahora**, sin haber programado nada. Esa es la clave de los Puntos de Función: medir lo que el sistema *hace*, no cómo está construido por dentro.

Ahí está toda la idea

Si no puedo medir el código, mido **lo que el sistema hace para el usuario** — y de ahí derivó cuánto trabajo cuesta construirlo.

Es la misma jugada de la analogía del puente: se miden atributos internos que *sí* puedo contar — pantallas, reportes, entidades — para derivar uno externo que no puedo ver todavía, como el esfuerzo en personas-mes.





ORIGEN

Puntos de Función

Allan Albrecht, IBM, 1979. Una medida del **tamaño funcional** del software: se cuenta lo que el sistema le ofrece al usuario, no cómo está construido por dentro.

En análisis

Se calcula con requerimientos, sin código.

Independiente

No depende del lenguaje ni la tecnología.

Comparable

Permite comparar proyectos distintos.

Antes de nombrar nada, un sistema

Funciones

- Registrar equipos
- Registrar partidos
- Buscar partido por fecha
- Actualizar datos de equipos
- Eliminar equipos
- Listado de equipos
- Reporte de equipos por rango de fechas
- Reporte de partidos

Entidades

- 4 entidades del dominio

El pedido del club

Tenemos una liga de fútbol amateur en Trelew y manejamos todo en papel. Necesitamos cargar los equipos y los partidos: quién jugó contra quién, la fecha y el resultado. Los datos de los equipos cambian seguido —cambia el delegado, cambia la cancha donde hacen de local— así que hay que poder corregirlos, y cuando un equipo se baja, sacarlo.

La gente llama para saber cuándo juega tal equipo, así que hay que poder buscar un partido por fecha. Y para las reuniones necesitamos el listado de equipos, un reporte de los equipos anotados en un rango de fechas, y un reporte de los partidos jugados.

¿Cuántos tipos de cosas hay acá?

Esas funciones no son todas del mismo tipo. Algunas cargan datos, otras muestran, otras buscan. Y las entidades son otra cosa distinta.

¿En cuántos grupos las separarían?

El sistema se parte en dos

Todo sistema hace dos cosas fundamentales: **procesa** y **guarda**. Sobre esa división se arman los cinco tipos de componentes que vamos a contar.

Funciones de transacción

Lo que el usuario **hace** con el sistema.

- **EI** — Entrada Externa
- **EO** — Salida Externa
- **EQ** — Consulta Externa

Funciones de datos

Lo que el sistema **guarda** o toma de otros.

- **ILF** — Archivo Lógico Interno
- **EIF** — Archivo de Interfaz Externo

Los cinco componentes en detalle

El — *External Input*

El usuario **mete** datos que cambian algo. Altas, bajas, modificaciones. Ejemplo: *Registrar equipo.*

EO — *External Output*

El sistema **entrega información elaborada**: reportes con totales, gráficos, listados calculados. Hay procesamiento en el medio.

EQ — *External Inquiry*

El usuario **pide y ve** datos sin cálculo y sin modificar nada. Ejemplo: *Buscar partido por fecha.*

ILF — *Internal Logical File*

Datos que **el sistema mantiene**. En la práctica, casi siempre una entidad del modelo de dominio.

EIF — *External Interface File*

Datos que el sistema **usa pero no mantiene**: los administra otro sistema.

La diferencia clave: EO vs. EQ

Es la confusión más frecuente en el conteo. La regla práctica es simple: **si hay una cuenta en el medio, es EO.**

EQ — *External Inquiry*

Recupera y muestra, sin procesar.

"Buscar el partido del 12 de marzo."

Sin cálculo · Sin modificación · Sin derivación

EO — *External Output*

Recupera **y además procesa**: totaliza, calcula, ordena por criterio derivado.

"Reporte de goles por equipo del último semestre."

ILF: "lógico" no es "físico"

ILF — *Internal Logical File* — y EIF — *External Interface File* — se cuentan desde el **punto de vista lógico**, no del sistema de archivos ni de la base de datos. Esto es una fuente frecuente de errores.

No contar tablas

Una entidad con sus tablas auxiliares puede ser **un solo ILF**. Contar tablas sobreestima.

Contar entidades lógicas

Lo que importa es qué **concepto del dominio** agrupa esos datos: Equipo, Partido, Jugador.

 Un archivo físico puede no ser ningún ILF. Una entidad con tablas relacionadas puede ser uno solo.

Ojo con la bibliografía

En los libros en castellano vas a encontrar las siglas traducidas. Son las mismas cosas. Acá usamos las inglesas, que son las del estándar IFPUG.

Sigla IFPUG (inglés)	Sigla en castellano
EI — <i>External Input</i>	EE — Entrada Externa
EO — <i>External Output</i>	SE — Salida Externa
EQ — <i>External Inquiry</i>	CE — Consulta Externa
ILF — <i>Internal Logical File</i>	ALI — Archivo Lógico Interno
EIF — <i>External Interface File</i>	AIE — Archivo de Interfaz Externo

Cómo se reconocen en la práctica

Esta tabla es una **guía, no una regla automática**. Siempre hay que mirar qué hace la función para el usuario, no cómo se llama.

Lo que dice el requerimiento	Suele ser
Insertar, Registrar, Actualizar, Eliminar	EI
Listar, Informe, Reporte (con cálculos)	EO
Buscar, Consultar, Ver (sin cálculo)	EQ
Entidad que el sistema mantiene	ILF
Datos que administra otro sistema	EIF

No todo pesa igual — la tabla de pesos

Cada componente se pondera según su complejidad. Esta tabla se fija **antes de empezar el conteo** y no cambia.

Componente	Simple	Medio	Complejo
EI — Entrada Externa	3	4	6
EO — Salida Externa	4	5	7
EQ — Consulta Externa	3	4	6
ILF — Archivo Lógico Interno	7	10	15
EIF — Archivo de Interfaz Externo	5	7	10

Los ILF pesan mucho más que las transacciones: el modelo de datos es el núcleo del sistema.

Ahora sí, a contar

Volvemos al sistema de fútbol del comienzo. Ya sabemos cómo se llama cada tipo de función y cuánto pesa cada una. Clasifiquen cada función, y después pesen.

Función	Tipo	Cuenta × Peso	PF	Complejidad
Registrar equipos	EI	1 × 4	4	Media
Registrar partidos	EI	1 × 4	4	Media
Actualizar datos de equipos	EI	1 × 4	4	Media
Eliminar equipos	EI	1 × 4	4	Media
Buscar partido por fecha	EQ	1 × 4	4	Media
Listado de equipos	EO	1 × 5	5	Media
Reporte por rango de fechas	EO	1 × 5	5	Media
Reporte de partidos	EO	1 × 5	5	Media
Entidades del sistema	ILF	4 × 10	40	Media

Resultado: Puntos de Función Sin Ajustar

75

PFSA

Puntos de Función Sin Ajustar del sistema de fútbol

40

De los ILF

4 entidades × 10 puntos cada una — más de la mitad del total

35

De transacciones

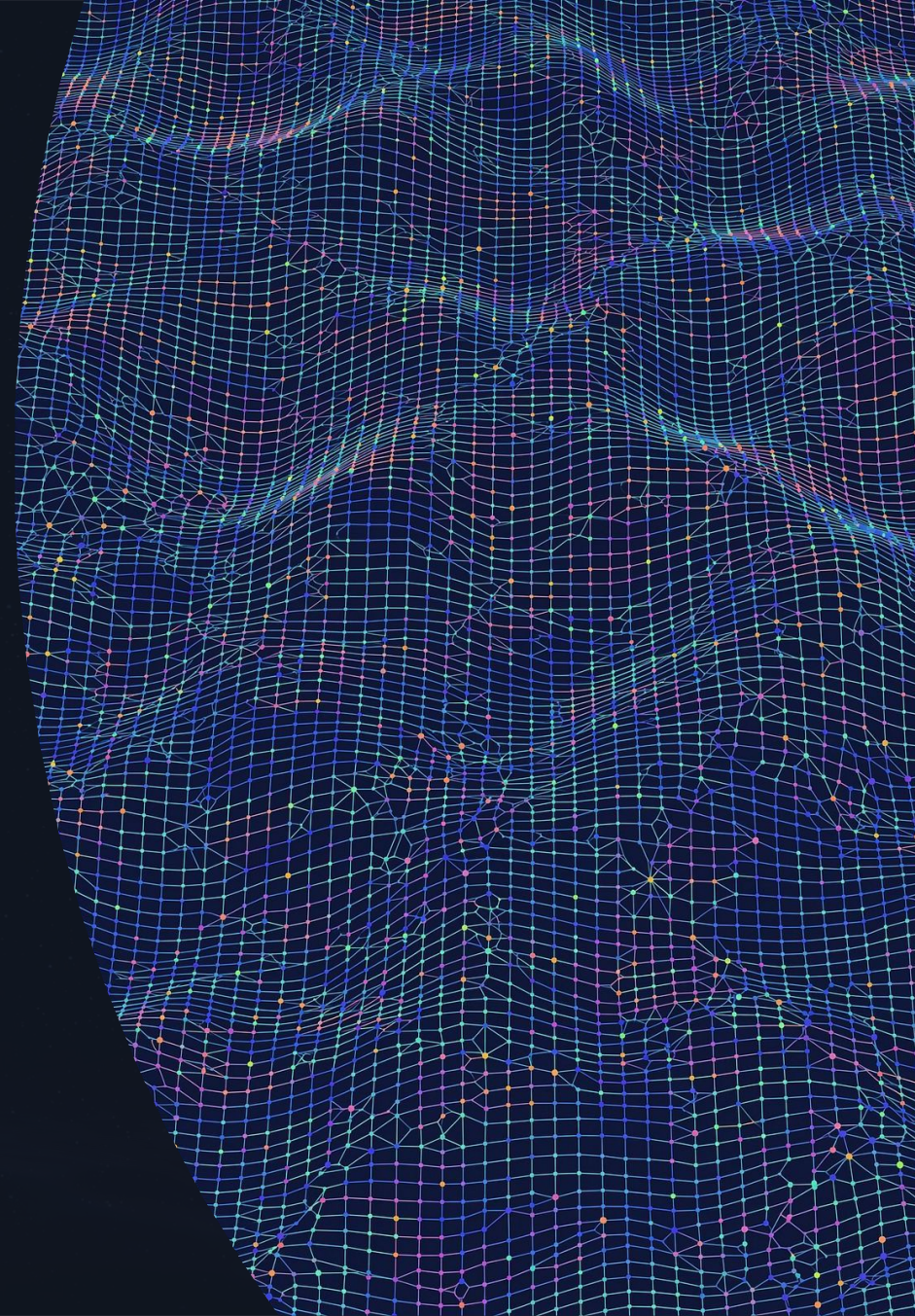
8 funciones de entrada, salida y consulta

Ya tenemos una medida del tamaño del sistema — y todavía no escribimos una línea de código.

FACTORES DE AJUSTE

Dos sistemas, los mismos PF. ¿Cuestan lo mismo?

Un sistema corre en una notebook para diez usuarios internos. El otro es distribuido, en seis sedes, con datos sensibles y respuesta en tiempo real. Ambos tienen 75 PF. La complejidad del entorno importa — y Albrecht lo sabía.



Los 14 factores de ajuste

Albrecht identificó **14 características** que hacen que un sistema cueste más o menos con la misma funcionalidad. No describen *qué hace* el sistema, sino **en qué condiciones tiene que hacerlo**.

#	Factor	#	Factor
1	Comunicación de datos	8	Actualización en línea
2	Procesamiento distribuido	9	Procesamiento complejo
3	Rendimiento	10	Reusabilidad
4	Uso intensivo de la configuración	11	Facilidad de instalación
5	Tasa de transacciones	12	Facilidad de operación
6	Entrada de datos en línea	13	Múltiples sitios
7	Eficiencia para el usuario final	14	Facilidad de cambio

Cómo se puntúa cada factor

Cada factor recibe un puntaje de **0 a 5** según cuánta influencia tiene en este sistema concreto. Lo que importa no es el número: es que se pueda **justificar**.

0 — Ninguna

No aplica al sistema.

1 — Incidental

Influencia menor.

2 — Moderada

Presente pero no dominante.

3 — Media

Influencia notable.

4 — Significativa

Condiciona el diseño.

5 — Esencial

Define el sistema.

La suma de los 14 factores (ΣFi) queda entre **0 y 70**.

La fórmula del ajuste

$$PFA = PFSA \times [0,65 + (0,01 \times \Sigma Fi)]$$

Variables

- **PFSA** — Puntos de Función Sin Ajustar (el conteo)
- **ΣFi** — Suma de los 14 factores (0 a 70)
- **PFA** — Puntos de Función Ajustados (resultado final)

Rango del factor de ajuste

- Con $\Sigma Fi = 0 \rightarrow$ factor = **0,65**
- Con $\Sigma Fi = 35 \rightarrow$ factor = **1,00**
- Con $\Sigma Fi = 70 \rightarrow$ factor = **1,35**

El ajuste solo puede mover el resultado **$\pm 35\%$** .

El ajuste del sistema de fútbol

Con **PFSA = 75** y un **ΣFi = 32** (sistema sencillo, sin distribución ni requisitos especiales de rendimiento):

$$PFA = 75 \times [0,65 + (0,01 \times 32)] = 75 \times 0,97 \approx 73$$

❏ Lo que de verdad define el tamaño es **el conteo**, no los factores. Invertí el tiempo en contar bien — discutir si un factor va en 3 o en 4 mueve muy poco el resultado final.

Ya tenemos el tamaño. Falta el trabajo.

Los PF miden **cuán grande** es el sistema. Todavía no dicen cuántos meses ni cuánto dinero. Para eso hay que convertir — y ahí entran los **datos históricos** de la organización.



De PF a líneas de código

Cada lenguaje necesita más o menos código para implementar la misma funcionalidad. Estos valores son **órdenes de magnitud** de tablas de la industria (Jones, QSM). Los datos propios de la organización siempre son mejores.

Lenguaje	LDC por PF (aprox.)
Assembler	~320
C	~128
Java / C#	~55
JavaScript / TypeScript	~47
Python	~30
SQL	~15
Generadores de código	~15

De PF a esfuerzo

La vía corta usa la **productividad del equipo**, medida en PF por persona-mes. Este dato viene de proyectos anteriores de la organización.

Ejemplo concreto

Si el equipo produce **6,5 PF por persona-mes** y el sistema tiene **372 PF**:

$$372 \div 6,5 \approx 57 \text{ personas-mes}$$

La otra vía

También se puede ir por las líneas de código:

PF → LDC → COCOMO → esfuerzo

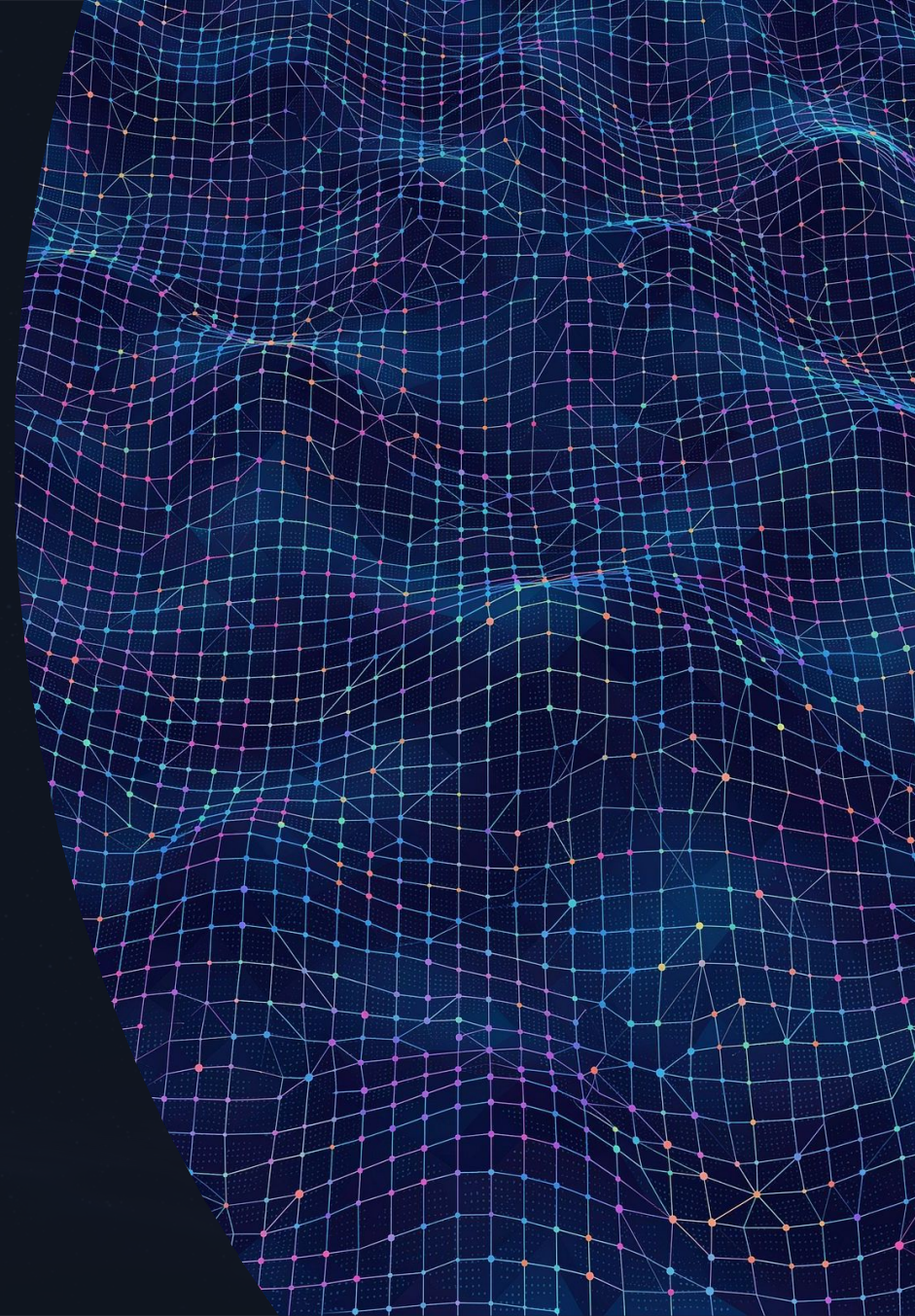
Sirve cuando hay datos históricos de LDC pero no de PF. Da un resultado similar, no idéntico.

LEY DE BROOKS

El esfuerzo no se divide por la gente disponible

57 personas-mes **no son** 57 personas trabajando un mes. Sumar gente a un proyecto atrasado lo atrasa más: hay que capacitarlos, y cada persona nueva multiplica los canales de comunicación.

⚠ El plazo tiene un mínimo físico que no se puede comprimir sumando recursos.



De esfuerzo a costo

Costo = esfuerzo (personas-mes) × costo mensual **cargado** de una persona.

"**Cargado**" quiere decir el costo real y completo: no solo el sueldo neto, sino también:



Cargas sociales

Aportes patronales y contribuciones sobre el salario bruto.



Infraestructura y licencias

Hardware, servicios cloud, herramientas de desarrollo.

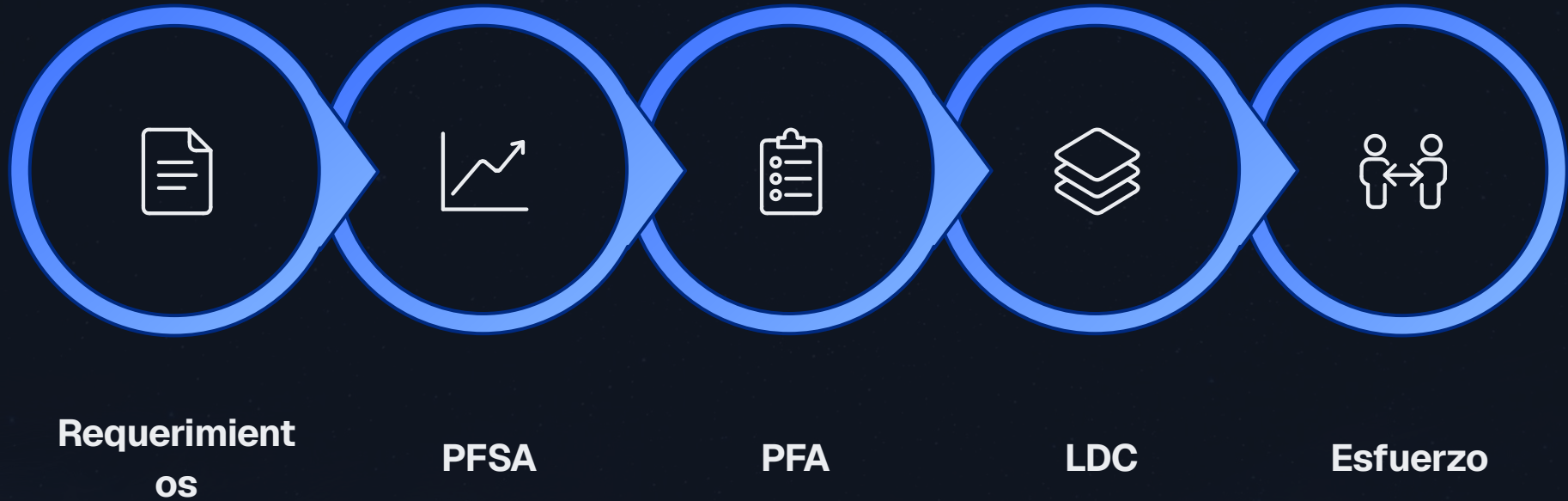


Tiempo no facturable

Vacaciones, capacitaciones, reuniones internas, días de enfermedad.

La cadena completa

Cada flecha suma incertidumbre. Al final no se da un número: se da un **rango**, y se explica de dónde salió cada paso.



CONCEPTO CLAVE

Un número no es un compromiso

Estimar

Calcular cuánto *podría* costar, basándose en información disponible hoy.

Es un acto técnico. Lo hacen los ingenieros.

Comprometerse

Prometer una fecha y un precio. Es un acto de gestión y negocio.

Lo deciden otras personas, con otra información.

Confundir los dos actos — y dejar que el cliente interprete una estimación como un compromiso — es el origen de la mitad de los proyectos que fracasan.

Lo que PF no hace

Conocer los límites de una herramienta es parte de saber usarla bien.

No mide calidad

Un sistema de 300 PF puede estar muy bien o muy mal construido. Los PF no distinguen entre los dos.

No mide dificultad algorítmica

Un motor de cálculo complejo puede pesar poco en PF. La complejidad interna no queda capturada.

No sirve para comparar personas

Medir la productividad individual en PF/mes es un error metodológico.

Las críticas honestas

Los Puntos de Función tienen limitaciones reales. Conocerlas no los invalida — los pone en perspectiva.



Cuesta trabajo contarlos

Un conteo serio lleva tiempo, y los proyectos suelen ir ajustados. La tentación de saltar pasos es real.



El conteo depende de quién cuenta

Dos analistas distintos pueden obtener valores diferentes. Hay que fijar criterios claros y mantenerlos en todo el equipo.



Pierde precisión en sistemas de cálculo intensivo

Sistemas con mucha lógica y poca interacción con el usuario quedan subrepresentados en PF.

Aun con todo eso: se puede calcular en la etapa de análisis. **Ninguna otra medida de tamaño puede decir lo mismo.**

Equivocarse es parte del método

Una organización que estima y nunca revisa

No mejora nunca. Sus números son siempre igual de malos, o igual de afortunados.

Una organización que registra sus errores

Calibra sus factores de productividad. En dos o tres proyectos, sus estimaciones son defendibles. En cinco, son confiables.

Ninguna estimación es exacta. Lo que la vuelve útil es **quedar escrita** y después **compararse contra lo que pasó**. Ese ciclo es el núcleo de la mejora continua en ingeniería de software.



EL HILO EN UNA FRASE

No podemos medir el código que todavía no escribimos.

Pero sí podemos **contar lo que el sistema va a hacer** — y con eso, y con lo que nos pasó antes, poner un número que se pueda defender.

Eso es Puntos de Función. Eso es estimación.